



TUGAS TUTORIAL ONLINE 1

NAMA : AKHMAD MUDEHIR
NIM : 055745857
PROGRAM STUDI : SISTEM INFORMASI
FAKULTAS : SAINS DAN TEKNOLOGI
UPBJJ/UT DAERAH : JAKARTA
NAMA/KODE MATA KULIAH : ALGORITMA DAN PEMROGRAMAN / STSI4102
KODE KELAS : 56

SOAL TUGAS

1. Buat Rangkuman yang bahan dan materinya dari Internet tentang:
 - a. Algoritma dan Dunia Pemrograman Komputer, dengan Spasi 1,5 dibuat pada Microsoft Word, dengan Daftar Pustaka Minimal 5
 - b. Sejarah Java dan perkembangannya sampai saat ini, cari referensi di internet berkaitan dengan pemrograman Java dan Software-software yang dibuat dengan Bahasa pemrograman Java minimal 2 software.
2. Buatlah Algoritma dalam bentuk flowchart dan pseudocode kasus berikut:

Pada sebuah perusahaan dengan tiga golongan Karyawan, yaitu:

- a. Jika Golongan A: Gaji Rp. 5.000.000
- b. Jika Golongan B: Gaji Rp. 6.500.000
- c. Jika Golongan C: Gaji Rp. 9.500.000

Jika karyawan tersebut lembur, maka mereka dibayar per-Jam dengan ketentuan sebagai berikut:

1. Jika Karyawan lembur 1 Jam maka gaji lemburnya 30% dari Gaji Pokok
2. Jika Karyawan lembur 2 Jam maka gaji lemburnya 32% dari Gaji Pokok
3. Jika Karyawan lembur 3 Jam maka gaji lemburnya 34% dari Gaji Pokok
4. Jika Karyawan lembur 4 Jam maka gaji lemburnya 36% dari Gaji Pokok
5. Jika Karyawan lembur ≥ 5 Jam maka gaji lemburnya 38% dari Gaji Pokok

Inputan pada Flowchart dan Pseudocode adalah Golongan dan Jam Lembur.

Output Jumlah penghasilan: Gaji Golongan + Gaji Lembur.

BAGIAN 1: Algoritma dan Dunia Pemrograman Komputer serta Sejarah dan Perkembangan Bahasa Pemrograman Java

Bab I: Algoritma dan Dunia Pemrograman Komputer

Pengertian Algoritma

Pada dasarnya, algoritma dapat dianalogikan seperti sebuah resep masakan. Resep tersebut menyediakan serangkaian langkah-langkah yang terstruktur dan harus diikuti secara berurutan untuk menghasilkan hidangan yang diinginkan. **Secara sederhana, Algoritma bisa diartikan sebagai langkah-langkah yang tersusun dan teratur serta masuk akal (logis) yang bertujuan untuk memecah berbagai permasalahan yang spesifik.** Jika urutan langkah diubah atau ada langkah yang dilewati, maka hasil akhirnya mungkin tidak akan sesuai dengan apa yang diinginkan. Di dunia komputasi, komputer juga pada dasarnya adalah mesin yang hanya bisa menjalankan perintah. Agar komputer tersebut dapat melakukan tugas-tugas yang kompleks, seperti mengirim email, menjalankan game, membuat desain grafis, dll, maka komputer akan selalu membutuhkan algoritma yang jelas dan efisien sebagai panduannya.

Jika kita ilustrasikan Algoritma adalah jantung dari hampir semua aplikasi perangkat lunak yang kita gunakan sehari-hari, mulai dari cara mesin pencari Google menentukan peringkat hasil pencarian suatu website (SEO) hingga bagaimana fitur GPS yang terdapat pada Google Maps mampu menentukan rute tercepat dalam perjalanan ke tempat tujuan. Istilah "**algoritma**" itu sendiri justru memiliki akar sejarah yang mendalam, berasal dari nama seorang matematikawan Persia abad ke-9, **Abu Ja'far Muhammad Ibnu Musa al-Khwarizmi**, yang karyanya memperkenalkan metode sistematis untuk menyelesaikan persamaan linear dan kuadrat. Yaitu *Al-Kitab al-Mukhtasar fi Hisab al-Jabr wal-Muqabala*.

Simbiosis Algoritma dan Program: Dari Logika ke Implementasi

Sangat penting untuk memahami perbedaan sekaligus hubungan simbiosis antara algoritma dan program komputer. **Algoritma adalah konsep, logika, atau kerangka berpikir**, algoritma bisa dianggap sebuah cetak biru (blueprint) yang bersifat independen dari bahasa pemrograman manapun. Di sisi lain, **program komputer adalah implementasi konkret** dari algoritma tersebut yang ditulis dalam sintaksis bahasa pemrograman tertentu (seperti Java, Python, atau C++) agar dapat dieksekusi dan dibaca oleh mesin.

Hubungan ini dapat dipahami sebagai sebuah "tangga abstraksi" dalam proses rekayasa perangkat lunak. Proses pengembangan sebuah solusi komputasi biasanya bergerak dari tingkat abstraksi tertinggi ke yang terendah:

1. **Pemahaman Konseptual (Tingkat Atas):** Ini adalah tahap di mana masalah dianalisis dan dipahami. Hasilnya adalah pemahaman mendalam tentang apa yang harus diselesaikan, seperti yang dituangkan dalam rangkuman teoretis ini.
2. **Desain Terstruktur (Tingkat Menengah):** Di sini, logika solusi dirancang. Algoritma diekspresikan dalam format yang masih bisa dibaca manusia tetapi sudah lebih terstruktur, seperti *pseudocode* (representasi tekstual) dan *flowchart* (representasi visual). Tahap ini memungkinkan perancang untuk memverifikasi logika tanpa terikat pada detail sintaksis bahasa pemrograman.
3. **Implementasi Kode (Tingkat Bawah):** Pada tahap ini, desain algoritma dari *pseudocode* atau *flowchart* diterjemahkan menjadi kode program yang sebenarnya menggunakan bahasa pemrograman yang dipilih.

Memahami alur kerja ini, dari konsep ke desain, lalu ke implementasi adalah kunci untuk menjadi seorang programmer atau analis sistem yang efektif. Ini memastikan bahwa solusi yang dibangun tidak hanya berfungsi, tetapi juga didasarkan pada logika yang solid, terstruktur, dan telah diverifikasi.

Anatomi dan Cara Kerja Universal Algoritma

Setiap algoritma, terlepas dari kompleksitasnya, beroperasi berdasarkan model tiga tahap yang mendasar: **Input -> Proses -> Output**.

- **Input (Masukan):** Ini adalah data awal, informasi, atau perintah yang diberikan kepada algoritma untuk diolah. Sebuah algoritma dapat memiliki nol atau lebih input yang terdefinisi dengan baik. Contohnya, dalam algoritma pencarian di toko online, inputnya adalah nama produk yang diketik oleh pengguna.
- **Proses:** Ini adalah inti dari algoritma, di mana serangkaian langkah-langkah komputasi, operasi logika, perbandingan, dan pengambilan keputusan dieksekusi secara berurutan untuk mengubah input menjadi hasil yang diinginkan. Dalam contoh toko online, prosesnya bisa melibatkan pencocokan input pengguna dengan database produk, mengurutkan hasilnya berdasarkan relevansi, dan memfilter berdasarkan ketersediaan.
- **Output (Keluaran):** Ini adalah hasil akhir atau solusi yang dihasilkan oleh algoritma setelah tahap proses selesai. Algoritma harus menghasilkan setidaknya satu output yang relevan dan merupakan jawaban dari masalah yang diberikan. Dalam contoh kita, outputnya adalah daftar produk yang relevan yang ditampilkan kepada pengguna.

Model sederhana ini mendasari semua operasi komputasi, dari kalkulator saku hingga sistem kecerdasan buatan yang kompleks.

Karakteristik Kunci Algoritma yang Baik

Tidak semua rangkaian instruksi dapat dianggap sebagai algoritma yang baik. Menurut ilmuwan komputer legendaris Donald Knuth, sebuah algoritma yang berkualitas harus memenuhi lima kriteria ketat:

1. **Finiteness (Keterbatasan):** Algoritma harus dijamin berhenti setelah menjalankan sejumlah langkah yang terbatas. Ia tidak boleh berjalan selamanya dalam sebuah *infinite loop*. Karakteristik ini memastikan bahwa program akan selalu memberikan hasil dalam waktu yang wajar.
2. **Definiteness (Kepastian):** Setiap langkah dalam algoritma harus didefinisikan secara tepat, jelas, dan tidak ambigu (tidak bermakna ganda). Jika dua programmer yang berbeda mengimplementasikan algoritma yang sama dengan input yang sama, mereka harus mendapatkan output yang identik. Ini krusial untuk konsistensi dan keandalan.
3. **Input (Masukan):** Sebuah algoritma harus memiliki nol atau lebih masukan yang terdefinisi dengan jelas. Kita harus tahu persis data apa yang dibutuhkan oleh algoritma untuk dapat bekerja.

4. **Output (Keluaran):** Sebuah algoritma harus menghasilkan setidaknya satu keluaran. Keluaran ini harus merupakan solusi dari masalah yang ingin dipecahkan.
5. **Effectiveness (Efektivitas):** Setiap instruksi dalam algoritma harus cukup mendasar sehingga dapat dieksekusi secara efektif. Artinya, setiap langkah harus praktis, dapat diimplementasikan, dan dapat diselesaikan dalam waktu yang wajar.

Karakteristik ini bukan sekadar teori akademis, melainkan memiliki dampak langsung pada dunia bisnis dan ekonomi. Sebuah algoritma yang tidak efisien (*effectiveness*), ketika dijalankan pada skala besar seperti di Google atau Amazon, akan memerlukan lebih banyak daya komputasi (server, listrik), yang berarti biaya operasional yang lebih tinggi. Algoritma yang tidak pasti (*definiteness*) akan menghasilkan *bug* dan *error*, yang tidak hanya mahal untuk diperbaiki tetapi juga dapat merusak pengalaman pengguna dan reputasi perusahaan. Dengan demikian, merancang "algoritma yang baik" yang memenuhi kelima kriteria ini bukan hanya tantangan teknis, tetapi juga merupakan sebuah aset bisnis yang strategis.

Peran dan Fungsi Vital Algoritma dalam Pemrograman

Algoritma memainkan peran sentral dalam pengembangan perangkat lunak dengan berbagai fungsi krusial, yang dapat dikelompokkan sebagai berikut:

- **Pemecahan Masalah:** Fungsi paling fundamental dari algoritma adalah menyediakan kerangka kerja yang logis dan sistematis untuk memecahkan masalah. Ini mengubah pendekatan "coba-coba" menjadi proses rekayasa yang terstruktur.
- **Optimisasi Kinerja:** Algoritma yang efisien dirancang untuk meminimalkan penggunaan sumber daya, terutama waktu eksekusi (*time complexity*) dan penggunaan memori (*space complexity*). Ini membuat aplikasi berjalan lebih cepat, lebih responsif, dan mampu menangani lebih banyak pengguna secara bersamaan.
- **Menyederhanakan Kompleksitas dan Strukturisasi Kode:** Algoritma membantu memecah masalah yang besar dan kompleks menjadi bagian-bagian yang lebih kecil dan lebih mudah dikelola. Ini memungkinkan programmer untuk menulis kode yang lebih terstruktur, logis, dan modular.
- **Meningkatkan Keterbacaan dan Pemeliharaan:** Dengan alur yang jelas, algoritma membuat kode program lebih mudah dibaca dan dipahami oleh programmer lain. Hal ini sangat penting dalam proyek tim dan untuk pemeliharaan perangkat lunak jangka panjang, karena memudahkan proses pencarian dan perbaikan kesalahan (*debugging*).

- **Reusabilitas dan Skalabilitas:** Algoritma yang dirancang dengan baik seringkali dapat digunakan kembali (*reusable*) untuk menyelesaikan masalah serupa di proyek lain. Ini juga mendukung skalabilitas, di mana aplikasi dapat dikembangkan untuk menangani volume data atau pengguna yang lebih besar di masa depan tanpa harus merombak seluruh sistem.

Klasifikasi Algoritma

Untuk menunjukkan wawasan yang lebih luas, penting untuk mengetahui bahwa algoritma dapat diklasifikasikan ke dalam beberapa paradigma atau jenis berdasarkan pendekatan pemecahan masalahnya. Beberapa yang paling umum adalah:

- **Algoritma Greedy:** Pendekatan ini membuat pilihan yang tampaknya paling optimal pada setiap langkah dengan harapan akan mengarah pada solusi optimal secara global. Contoh klasik adalah aplikasi peta seperti Google Maps yang memilih rute terpendek di setiap persimpangan untuk menemukan jalur tercepat secara keseluruhan.
- **Divide and Conquer:** Paradigma ini bekerja dengan memecah masalah besar menjadi beberapa sub-masalah yang lebih kecil dan sejenis, menyelesaikan setiap sub-masalah secara independen (seringkali secara rekursif), dan kemudian menggabungkan solusi-solusi tersebut untuk mendapatkan solusi akhir. Algoritma pengurutan data yang sangat efisien seperti *Merge Sort* dan *Quick Sort* adalah contoh utama dari pendekatan ini.
- **Algoritma Rekursif:** Ini adalah jenis algoritma di mana sebuah fungsi memanggil dirinya sendiri untuk menyelesaikan masalah. Pendekatan ini sangat elegan untuk masalah yang memiliki substruktur berulang, seperti menghitung nilai faktorial ($n!$) atau menghasilkan deret Fibonacci.

Daftar Pustaka (Bab I)

1. Alliana, V., Yahfizham, Y. (2024). Analisis Penerapan Dan Fungsi Algoritma Pemrograman Pada Komputer. *Jurnal Riset Rumpun Matematika dan Ilmu Pengetahuan Alam (JURRIMIPA)*. Diakses pada 20 Oktober 2025 dari <https://prin.or.id/index.php/JURRIMIPA/article/view/2401/2240>

2. UHAMKA. (2024). *Penemu Ilmu Aljabar yang Jenius dari Dunia Arab, Ini Sosoknya*. Diakses pada 20 Oktober 2025 dari <https://www.uhamka.ac.id/berita/Penemu-Ilmu-Aljabar-yang-Jenius-dari-Dunia-Arab--Ini-Sosoknya>
 3. Cibro, E., & Yahfizham, Y. (2024). Mengidentifikasi Dasar Algoritma Pemrograman. *Jurnal Ilmiah Sistem Informasi Dan Ilmu Komputer*. Diakses pada 20 Oktober 2025 dari <https://journal.sinov.id/index.php/juisik/article/view/763/700>
 4. Faradilla. (2025). Apa Itu Algoritma Pemrograman? Pahami Arti, Fungsi, dan Jenis-Jenisnya. Diakses pada 20 Oktober 2025 dari <https://www.hostinger.com/id/tutorial/apa-itu-algoritma-pemrograman>
 5. Khairiah, I. & Yahfizham, Y. (2023). Algoritma Pemrograman : Studi Pustaka Pemahaman Algoritma Pemrograman. *Konstanta : Jurnal Matematika dan Ilmu Pengetahuan Alam*. Diakses pada 20 Oktober 2025 dari <https://ifrelresearch.org/index.php/konstanta-widyakarya/article/view/1673/1647>
 6. Siregar, R. & Yahfizham, Y. (2023). Konsep Dasar Algoritma Pemrograman. *Jurnal Arjuna: Publikasi Ilmu Pendidikan, Bahasa dan Matematika*, <https://journal.aripi.or.id/index.php/Arjuna/article/view/333/353>
 7. Fikriansyah. I. (2023). Algoritma Adalah: Pengertian, Ciri-ciri, dan Berbagai Jenisnya. Diakses pada 20 Oktober 2025 dari <https://www.detik.com/edu/detikpedia/d-6817970/algoritma-adalah-pengertian-ciri-ciri-dan-berbagai-jenisnya>
-

Bab II: Sejarah dan Perkembangan Bahasa Pemrograman

Java

Sejarah Java: Proyek Hijau dan Filosofi "Tulis Sekali, Jalankan di Mana Saja"

Sejarah Java dimulai pada awal tahun 1990-an di Sun Microsystems melalui sebuah proyek rahasia yang dikenal sebagai "**The Green Project**". Dipimpin oleh trio visioner, **James Gosling**, Patrick Naughton, dan Mike Sheridan, proyek ini awalnya bertujuan untuk menciptakan teknologi bagi perangkat elektronik konsumen generasi berikutnya, seperti televisi interaktif dan *set-top box*. Tim ini menyadari bahwa perangkat-perangkat tersebut

menggunakan berbagai macam CPU, sehingga mereka membutuhkan bahasa pemrograman yang tidak terikat pada satu arsitektur perangkat keras tertentu (*architecture-neutral*).

Bahasa yang mereka ciptakan awalnya diberi nama "**Oak**", terinspirasi dari pohon ek yang kokoh di luar jendela kantor James Gosling. Namun, karena nama tersebut sudah menjadi merek dagang perusahaan lain, tim tersebut kemudian menggantinya menjadi "**Java**". Nama ini konon tercetus saat mereka sedang menikmati kopi di sebuah kafe, terinspirasi dari kopi Jawa (Java coffee) yang populer.

Kelahiran Java dan filosofi intinya, "**Write Once, Run Anywhere**" (WORA), bukanlah sebuah kebetulan, melainkan sebuah reaksi cerdas terhadap masalah besar yang melanda industri perangkat lunak pada saat itu. Sebelum Java, bahasa pemrograman seperti C++ sangat bergantung pada platform. Sebuah program yang ditulis untuk Windows harus ditulis ulang dan dikompilasi ulang secara signifikan agar bisa berjalan di sistem Mac atau Unix, sebuah proses yang sangat mahal dan memakan waktu.

Solusi jenius yang ditawarkan Java adalah **Java Virtual Machine (JVM)**. Alih-alih mengkompilasi kode sumber langsung ke bahasa mesin yang spesifik untuk satu platform, *compiler* Java mengubahnya menjadi format perantara yang disebut *bytecode*. *Bytecode* ini kemudian dapat dijalankan di platform manapun yang telah terinstal JVM. JVM bertindak sebagai penerjemah yang mengonversi *bytecode* universal menjadi instruksi mesin lokal secara *real-time*. Inovasi ini secara drastis mengurangi biaya dan waktu pengembangan perangkat lunak lintas platform, dan secara kebetulan, sangat cocok untuk era internet yang baru lahir, di mana aplikasi perlu diakses dari berbagai jenis perangkat dan sistem operasi di seluruh dunia.

Evolusi Java: Dari JDK 1.0 hingga Era Modern

Perkembangan Java adalah sebuah perjalanan evolusi yang berkelanjutan, ditandai dengan rilis-rilis utama yang secara konsisten menambahkan fungsionalitas, meningkatkan kinerja, dan memperkuat keamanan.

- **JDK 1.0 (Januari 1996):** Rilis publik pertama yang menandai kemunculan Java di panggung dunia. Versi ini meletakkan fondasi dengan menyediakan paket-paket inti

seperti `java.lang` (kelas dasar), `java.io` (input/output), dan **Abstract Window Toolkit (AWT)** untuk membangun antarmuka pengguna grafis (GUI) sederhana.

- **JDK 1.1 (Februari 1997):** Sebuah pembaruan signifikan yang memperkenalkan konsep **JavaBeans** (komponen perangkat lunak yang dapat digunakan kembali), **JDBC (Java Database Connectivity)** untuk interaksi dengan database, dan **RMI (Remote Method Invocation)** untuk komputasi terdistribusi.
- **J2SE 1.2 "Playground" (Desember 1998):** Dianggap sebagai rilis monumental yang melipatgandakan ukuran platform. Versi ini memperkenalkan **Swing**, sebuah *toolkit* GUI yang jauh lebih canggih dan fleksibel daripada AWT. Selain itu, **Collections Framework** diperkenalkan untuk menyediakan struktur data yang kuat dan efisien. Ini juga merupakan versi pertama yang menyertakan **Just-In-Time (JIT) compiler**, yang secara dramatis meningkatkan kinerja eksekusi program Java. Sejak rilis ini, platform ini dikenal sebagai Java 2.
- **J2SE 5.0 "Tiger" (September 2004):** Rilis ini membawa perubahan besar pada sintaksis bahasa itu sendiri, membuatnya lebih ekspresif dan aman. Fitur-fitur utamanya meliputi **Generics** (untuk keamanan tipe data dalam koleksi), **Annotations** (metadata untuk kode), *Autoboxing/Unboxing* (konversi otomatis antara tipe primitif dan objek), dan *Enhanced for-loop*.
- **Java SE 6 "Mustang" (Desember 2006):** Fokus utama rilis ini adalah pada peningkatan kinerja, stabilitas, dan integrasi. Versi ini menyertakan dukungan untuk bahasa skrip, peningkatan dramatis pada kinerja Swing, dan perbaikan pada JVM.
- **Era Modern (Java 8, 11, 17, 21 sebagai LTS):** Java terus berevolusi dengan siklus rilis yang lebih cepat. Versi-versi *Long-Term Support* (LTS) seperti Java 8, 11, 17, dan 21 menjadi standar industri. Java 8 (2014) merevolusi pemrograman Java dengan **Lambda Expressions** dan **Stream API**, memungkinkan gaya pemrograman fungsional. Rilis yang lebih baru seperti Java 21 terus berinovasi dengan fitur-fitur seperti **Virtual Threads**, yang dirancang untuk menyederhanakan dan meningkatkan skala aplikasi konkurensi tinggi.

Implementasi Bahasa Java pada Aplikasi Dunia Nyata Berbasis Java

Jika algoritma adalah DNA dari sebuah produk digital, maka bahasa pemrograman seperti Java adalah "kendaraan" atau "mesin" yang mengeksekusi DNA tersebut pada skala global. Java menyediakan ekosistem yang matang, kuat, dan skalabel yang dibutuhkan oleh

perusahaan teknologi raksasa untuk menjalankan algoritma-algoritma kompleks yang menjadi inti dari model bisnis mereka. Berikut adalah dua contoh nyata.

- Contoh 1: Netflix - Arsitektur Backend untuk Streaming Skala Global

Hampir seluruh infrastruktur backend yang menopang layanan streaming Netflix dibangun di atas Java. Untuk menangani jutaan permintaan per detik dari pengguna di seluruh dunia, Netflix mengadopsi arsitektur *microservices*. Fungsionalitas monolitik yang besar dipecah menjadi ratusan layanan kecil yang independen (misalnya, layanan untuk manajemen akun, penagihan, rekomendasi, pemutaran video), di mana setiap layanan dibangun sebagai aplikasi Java menggunakan framework populer Spring Boot.

Arsitektur ini memungkinkan tim yang berbeda untuk mengembangkan, menerapkan, dan menskalakan layanan mereka secara mandiri. Untuk mengelola komunikasi yang kompleks antara klien (TV, ponsel) dan ratusan *microservices* ini, Netflix menggunakan **Federated GraphQL** sebagai *API Gateway*. Teknologi ini memungkinkan aplikasi klien untuk meminta semua data yang mereka butuhkan dalam satu panggilan jaringan, dan *gateway* secara cerdas akan mengumpulkan data dari berbagai *microservices* di *backend*. Netflix juga secara aktif mengadopsi versi Java terbaru untuk mendapatkan keuntungan kinerja, seperti migrasi ke Java 21 untuk memanfaatkan **Virtual Threads**, yang secara signifikan menyederhanakan penanganan permintaan bersamaan dalam jumlah besar dan meningkatkan skalabilitas sistem mereka.

- Contoh 2: Spotify - Mengelola Data dan Layanan untuk Industri Musik Digital

Serupa dengan Netflix, backend Spotify juga sangat bergantung pada Java untuk menangani pemrosesan data dalam skala masif. Java menjadi tulang punggung sistem yang mengelola katalog musik raksasa, data pengguna, playlist, dan yang terpenting, algoritma rekomendasi musik yang menjadi ciri khas Spotify. Keandalan, skalabilitas, dan keamanan yang ditawarkan oleh ekosistem Java sangat krusial untuk memastikan pengalaman mendengarkan musik yang lancar dan terpersonalisasi bagi ratusan juta penggunanya.

Spotify menggunakan Java dalam arsitektur berbasis *cloud* mereka untuk menjalankan layanan-layanan yang berinteraksi dengan database terdistribusi seperti Apache Cassandra, yang dirancang untuk menangani volume data yang sangat besar. Filosofi rekayasa di Spotify bahkan menekankan pentingnya penguasaan mendalam pada beberapa teknologi inti, dan Java adalah salah satu pilar utama dalam tumpukan teknologi *backend* mereka.

Daftar Pustaka (Bab II)

1. Diengcyber. (2022). Perkembangan Bahasa Pemrograman Java. Diakses 22 Oktober 2025 dari <https://diengcyber.com/perkembangan-bahasa-pemrograman-java/>
2. Meygitaheka (2019). Sejarah Perkembangan Java. Diakses 22 Oktober 2025, dari <https://awanmendung7.home.blog/2019/12/29/sejarah-perkembangan-java/>
3. Wallis. J. (2023). How Does Spotify Work? Spotify Tech Stack Explored. Diakses 22 Oktober 2025 dari <https://intuji.com/how-does-spotify-work-tech-stack-explored/>
4. Kothiyal. P. (2025). How Netflix Runs on Java: Modern Backend Engineering at Scale. Diakses 22 Oktober 2025 dari <https://talent500.com/blog/netflix-java-backend-architecture/>
5. Wikipedia. (2025). Java version history. Diakses 22 Oktober 2025 dari https://en.wikipedia.org/wiki/Java_version_history

BAGIAN 2: Membuat Algoritma dalam bentuk flowchart dan pseudocode menggunakan studi kasus Penggajian Karyawan

Analisis Sistematis Masalah

Sebelum merancang sebuah algoritma, langkah pertama yang paling krusial adalah melakukan analisis masalah secara sistematis. agar kita dapat memahami semua komponen, aturan, dan tujuan dari masalah yang akan kita selesaikan. supaya mencegah kesalahan logika di tahap-tahap selanjutnya.

Untuk kasus perhitungan gaji karyawan ini, analisisnya dapat diuraikan sebagai berikut:

- **Deklarasikan variabel:** Data yang akan diperlukan untuk menyimpan nilai
 - Golongan (Tipe: Char atau String, contoh: 'A', 'B', 'C').
 - TotalGaji (Tipe: Double)
 - GajiPokok (Tipe: Double, contoh: 5000000, 6500000, 9500000)
 - GajiLembur (Tipe: Double)
 - JamLembur (Tipe: Double atau Integer, contoh: 1, 2, 3).
- **Rumus:** Proses perhitungan/ kalkulasi untuk menentukan GajiLembur dan TotalGaji
 - $GajiLembur = PersentaseJamLembur * GajiPokok$
 - $TotalGaji = GajiPokok + GajiLembur$
- **Output:** Menampilkan hasil kalkulasi dari TotalGaji

Perancangan Algoritma menggunakan Visualisasi Tekstual Alur Logika dengan Pseudocode

Pseudocode adalah cara untuk menuliskan algoritma menggunakan bahasa yang sederhana dan informal, yang menyerupai kode pemrograman tetapi tidak terikat pada sintaksis bahasa tertentu. Ini berfungsi sebagai jembatan antara analisis masalah dan penulisan kode yang sebenarnya.

```
1  01. START
2  02. Deklarasi Variabel:
3  03.   Char Golongan
4  04.   Double TotalGaji, GajiPokok, GajiLembur,
5  05.   Integer JamLembur
6  06. Input Data:
7  07.   PRINT "Masukkan Golongan (A/B/C): "
8  08.   READ Golongan
9  09.   PRINT "Masukkan Jumlah Jam Lembur: "
10 10.   READ JamLembur
11 11. Tentukan Gaji Pokok Berdasarkan Golongan:
12 12. IF Golongan == "A" THEN
13 12.1. GajiPokok ← 5000000.00
14 13. ELSE IF Golongan == "B" THEN
15 13.1. GajiPokok ← 6500000.00
16 14. ELSE IF Golongan == "C" THEN
17 14.1. GajiPokok ← 8500000.00
18 15. ELSE
19 15.1. PRINT "Bukan Golongan A/B/C"
20 15.2. BREAK
21 16. END IF
22 17. Tentukan Gaji Lembur Berdasarkan Jam Lembur:
23 18. IF JamLembur >= 5 THEN
24 18.1. GajiLembur ← 0.38 * GajiPokok
25 19. ELSE IF JamLembur == 4 THEN
26 19.1. GajiLembur ← 0.36 * GajiPokok
27 20. ELSE IF JamLembur == 3 THEN
28 20.1. GajiLembur ← 0.34 * GajiPokok
29 21. ELSE IF JamLembur == 2 THEN
30 21.1. GajiLembur ← 0.32 * GajiPokok
31 22. ELSE IF JamLembur == 1
32 22.1. GajiLembur ← 0.30 * GajiPokok
33 23. ELSE
34 23.1. GajiLembur ← 0
35 24. END IF
36 25. Hitung Total Gaji:
37 26.   TotalGaji ← GajiPokok + GajiLembur
38 27. Cetak Hasil:
39 28.   PRINT "Total Gaji Karyawan: " TotalGaji
40 29. SELESAI
```

Perancangan Algoritma menggunakan Visualisasi Alur Logika dengan Flowchart

Flowchart adalah representasi visual dari sebuah algoritma. Ini menggunakan simbol-simbol standar untuk menggambarkan alur kontrol dan proses, membuatnya sangat efektif untuk memahami logika yang kompleks secara intuitif.

Simbol yang Digunakan:

- **Terminator (Oval):** Simbol untuk **Mulai** dan **Selesai**
- **Preparation (Segi Enam):** Simbol untuk mendeklarasikan variabel **Golongan**, **TotalGaji**, **GajiPokok**, **GajiLembur**, dan **JamLembur**
- **Input/Output (Jajar Genjang):** Simbol untuk memberikan nilai inputan pada variabel **Golongan** dan **JamLembur** serta output untuk menampilkan variabel **TotalGaji**
- **Process (Persegi Panjang):** Simbol proses kalkulasi dari $\text{GajiLembur} = \% \text{JamLembur} * \text{GajiPokok}$ dan $\text{TotalGaji} = \text{GajiPokok} + \text{GajiLembur}$
- **Decision (Belah Ketupat):** Simbol untuk pengambilan keputusan dengan 2 opsi (Iya/Tidak), Iya jika kondisinya terpenuhi, Tidak jika kondisinya tidak terpenuhi. Untuk menentukan variabel $\text{Golongan} == 'A'/'B'/'C'$ dan $\text{JamLembur} \geq 5$ atau $== 4$ atau $== 3$ atau $== 2$ atau $== 1$
- **Arrow (Panah):** Menunjukkan aliran kontrol dalam suatu proses

